Lecture 24

C Programming IV

CS 12 Team 25.2
A.Y. 2025–2026, 2nd Sem

Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
  - (we’re working on getting the other TV fixed…)
- **Recording?**
- Important announcements?

Dynamically allocated nested arrays

Motivating example

Task: Create a function `??? make_matrix(int rows, int cows)` that returns a pointer to a dynamically created allocated matrix

Motivating example

Approach 1: Single pointer to a single contiguous allocation for all levels

- Returns `void *` for `n` dimensions
- Single pointer for `n` dimensions

Approach 1: Single contiguous allocation

For two dimensions:

- Allocation:
  - `malloc(ROWS * COLS * sizeof(int))`
  - Return type: `void *`
- Indexing:
  - Type: `int (*container)[COLS] = ...`
  - `container[r][c] ==`
    - `container + (r * COLS) + c`

- How many cells in each row (`r` index)?

Approach 1: Single contiguous allocation

`void *`: Generic object pointer that holds address of any data type

Convert to a specific pointer type before dereferencing

Approach 1: Single contiguous allocation

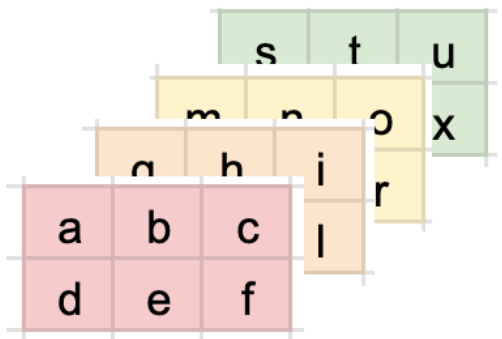
4 rows, 5 columns:

		0	1	2	3	4				
	0	a	b	c	d	e				
	1	f	g	h	i	j		addr	1000	
	2	k	l	m	n	o		var	**int *p**	
	3	p	q	r	s	t		data	**108**	
addr	108									
data	a	b	c	d	e	f	g	h	i	j
idx	0	1	2	3	4	5	6	7	8	9
data	k	l	m	n	o	p	q	r	s	t
idx	10	11	12	13	14	15	16	17	18	19

Approach 1: Single contiguous allocation

24/matrix.c

Approach 1: Single contiguous allocation



For `n` dimensions (`n=3` shown in figure):

Allocation:

- `malloc(Z_COUNT * Y_COUNT * X_COUNT * sizeof(int))`
- Return type: `void *`

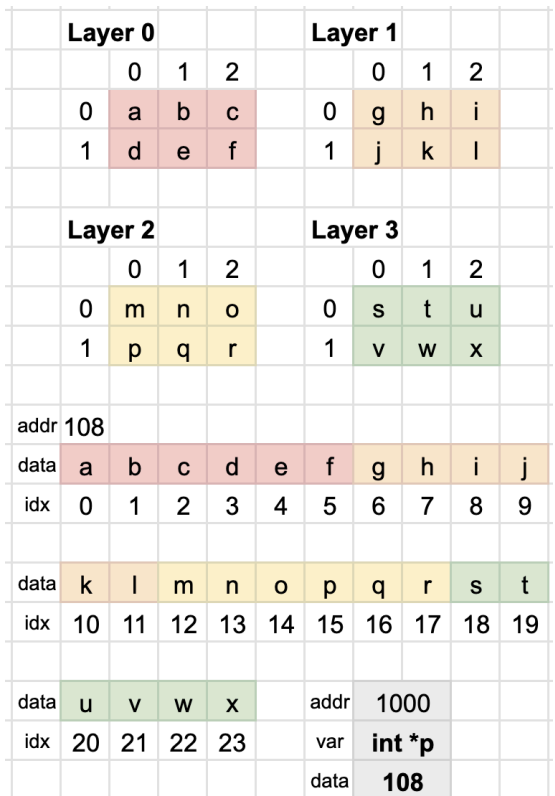
Indexing:

- Type: `int (*container)[Y_COUNT][X_COUNT] = ...`
- `container[z][y][x] ==`
  - `container + (z * Y_COUNT * X_COUNT) + (y * X_COUNT) + x`
  - How many cells in each `z` index?
  - How many cells in each `y` index?

Approach 1: Single contiguous allocation

4 layers, 2 rows, 3 columns:

Approach 1: Single contiguous allocation



	Layer 0			
	0	1	2
0	a	b	c
1	d	e	f

	Layer 1			
	0	1	2
0	g	h	i
1	j	k	l

	Layer 2			
	0	1	2
0	m	n	o
1	p	q	r

	Layer 3			
	0	1	2
0	s	t	u
1	v	w	x

addr	108									
data	a	b	c	d	e	f	g	h	i	j
idx	0	1	2	3	4	5	6	7	8	9
data	k	l	m	n	o	p	q	r	s	t
idx	10	11	12	13	14	15	16	17	18	19
data	u	v	w	x						
idx	20	21	22	23						

addr	1000
var	**int *p**
data	**108**

Motivating example

Approach 2: Pointers to contiguous allocations per level

- Returns `void **` for 2 dimensions
- Returns `void ***` for 3 dimensions
- `n` levels of pointers for `n` dimensions

Approach 2: Separate contiguous allocations

For two dimensions:

- Allocation (first level):
  - `malloc(ROWS * sizeof(void *))`
  - Return type: `void **`
- Allocation (second level):
  - `matrix[0] = malloc(COLS * sizeof(int))`
  - `matrix[1] = malloc(COLS * sizeof(int))`
  - `...`

- `matrix[ROWS-1] = malloc(COLS * sizeof(int))`

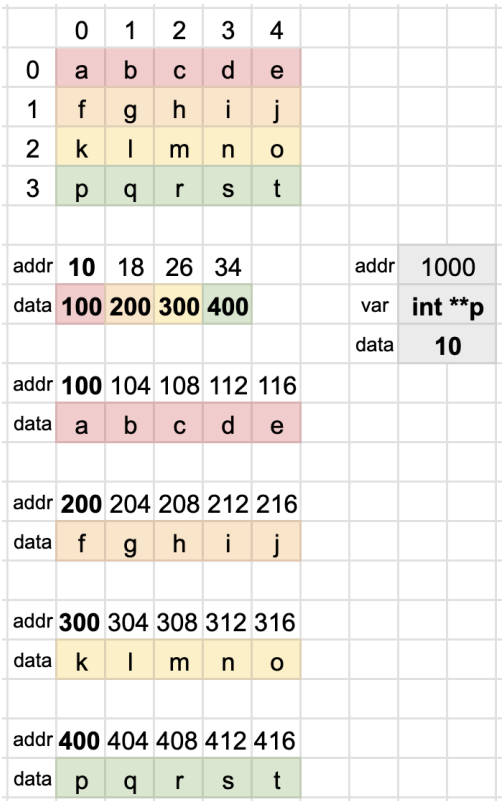
Approach 2: Separate contiguous allocations

- Indexing:
  - `container[r][c] == *(*(container + r) + c)`
    - How many cells in each row (r index)?

Approach 2: Separate contiguous allocations

4 rows, 5 columns:

Approach 2: Separate contiguous allocations



	0	1	2	3	4
0	a	b	c	d	e
1	f	g	h	i	j
2	k	l	m	n	o
3	p	q	r	s	t

addr	**10**	18	26	34	
data	**100**	**200**	**300**	**400**	

addr	1000
var	**int **p**
data	**10**

addr	**100**	104	108	112	116
data	a	b	c	d	e

addr	**200**	204	208	212	216
data	f	g	h	i	j

addr	**300**	304	308	312	316
data	k	l	m	n	o

addr	**400**	404	408	412	416
data	p	q	r	s	t

Approach 2: Separate contiguous allocations

24/matrix_noncontiguous.c

Approach 2: Separate contiguous allocations

For `n` dimensions (`n=3` shown below):

Allocation (first level):

- `malloc(Z_COUNT * sizeof(void **))`
- Return type: `void ***`

Approach 2: Separate contiguous allocations

- Allocation (second level):
  - `cont[0] = malloc(Y_COUNT * sizeof(void *))`
  - `cont[1] = malloc(Y_COUNT * sizeof(void *))`
  -
  - `cont[Z_COUNT-1] = ...`

Approach 2: Separate contiguous allocations

- Allocation (third level):
  - `cont[0][0] = malloc(X_COUNT * sizeof(int))`
  - `cont[0][1] = malloc(X_COUNT * sizeof(int))`
  - `...`
  - `cont[Z_COUNT-1][Y_COUNT-1] = ...`

Approach 2: Separate contiguous allocations

- Indexing:
  - Type: `int (*cont)[Y_COUNT][X_COUNT] = ...`
  - `cont[z][y][x] == *(*(*(cont + z) + y) + x)`

Approach 2: Separate contiguous allocations

4 layers, 2 rows, 3 columns:

Layer 0

	0	1	2
0	a	b	c
1	d	e	f

Layer 1

	0	1	2
0	g	h	i
1	j	k	l

addr	1000
var	**int ***p**
data	**10**

Layer 2

	0	1	2
0	m	n	o
1	p	q	r

Layer 3

	0	1	2
0	s	t	u
1	v	w	x

addr	10	18	26	34
data	**100**	**200**	**250**	**320**

addr	**100**	108	
data	**150**	**180**	

addr	**150**	154	158
data	a	b	c

addr	**180**	184	188
data	d	e	f

addr	**200**	208	
data	**400**	**500**	

addr	**400**	404	408
data	g	h	i

addr	**500**	504	508
data	j	k	l

addr	**250**	258	
data	**650**	**750**	

addr	**650**	654	658
data	m	n	o

addr	**750**	754	758
data	p	q	r

addr	**320**	328	
data	**910**	**932**	

addr	**910**	914	918
data	s	t	u

addr	**932**	936	940
data	v	w	x

Linked lists

Motivating example

Task: Make a program that repeatedly asks the user for integers; when `0` is given, all previously entered integers should be printed in the order they were entered

Why is the following solution incorrect?

Motivating example

```c
#include <stdio.h>

int main() {
    int nums[1000] = {}, count = 0, num;

    while (1) {
        printf("Enter an integer: ");
        scanf("%d", &num);

        if (num == 0) break;
        nums[count++] = num;  // Aside: nums[++count]
```

```
    }                                   // is incorrect!

    for (int i = 0; i < count; i++) {
        printf("Index %d: %d\n", i, nums[i]);
    }
}
```

Linked list

Linked list: Sequence of elements where each element links to the next

- head: Pointer to first element
  - Can be placed in a separate `struct` (see below)
- Each element points to either:
  - Next element
  - `NULL` (end of list)
- Dynamically allocates memory one element/node at a time

- Easily adjusts size unlike arrays, but slower accessing (later)

Linked list

Representation of a single element (node):

```
typedef struct node Node;
struct node {
    int data;
    Node *next;  // Recursive definition
};
```

Linked list

For convenience (optional):

```
typedef struct {
    Node *head;
    uint32_t size;
} LinkedList;
```

Static linked list construction

```
typedef struct node Node;
struct node {
    int data;
    Node *next;
};

typedef struct {
    Node *head;
    uint32_t size;
} LinkedList;
```

Static linked list construction

```
Node *make_node(int data, Node *next) {
    Node *ret = malloc(sizeof(Node));
    ret->data = data;
    ret->next = next;

    return ret;
}
```

Static linked list construction

`24/static_linked_list.c`

How do we print all linked list elements in order?

Traversal

Definitions for reference:

```
typedef struct node Node;
struct node {
    int data;
    Node *next;
};

typedef struct {
    Node *head;
    uint32_t size;
} LinkedList;
```

Traversal

Iterative approach (Python Tutor):

```
void print_linked_list(LinkedList *list) {
    Node *p = list->head;

    while (p != NULL) {
        printf("%d\n", p->data);
        p = p->next;
    }
}
```

Traversal

Recursive approach (Python Tutor):

```
void print_nodes(Node *node) {
    if (node == NULL) {
        return;
    }
    printf("%d\n", node->data);
    print_nodes(node->next);
}
void print_linked_list(LinkedList *list) {
    print_nodes(list->head);
}
```

Traversal

Self-check: Define `Node *get(LinkedList *list, int idx)` that returns pointer to node at "index" `idx` or `NULL` if not present; do both iterative and recursive versions

Insertion

Definitions for reference:

```
typedef struct node Node;
struct node {
    int data;
    Node *next;
};

typedef struct {
    Node *head;
    uint32_t size;
} LinkedList;
```

```
Node *make_node(int data, Node *next) {
    Node *ret = malloc(sizeof(Node));
    ret->data = data;
    ret->next = next;

    return ret;
}
```

Insertion

Insert before head (Python Tutor):

```
void prepend(LinkedList *list, int data) {
    Node *old_head = list->head;
    Node *new_head = make_node(data, old_head);

    list->head = new_head;
    list->size++;
}
```

Insertion

Insert after “tail” (Python Tutor):

```
void append(LinkedList *list, int data) {
    list->size++;

    if (list->head == NULL) {
        list->head = make_node(data, NULL);
        return;
    }
```

```
    Node *t = list->head;

    while (t->next != NULL) {
        t = t->next;
    }

    t->next = make_node(data, NULL);
}
```

Insertion

Self-check: Define `Node *insert_at(LinkedList *list, int idx, Node *new)` that inserts new at "index" `idx` (or end of list)

Deletion and more in CS 32